

SANTIAGO ENRIQUE VILLABONA APONTE



**TRABAJO DE GRADO PRESENTADO PARA OPTAR AL
TÍTULO DE**

INGENIERO DE SISTEMAS E INFORMÁTICA

**DESARROLLO DE UNA APLICACIÓN WEB PARA LA GESTIÓN
INTEGRADA DE INVENTARIO, VENTAS Y CLIENTES EN PEQUEÑOS
NEGOCIOS**

**ESCUELA DE INGENIERÍAS
INGENIERÍA DE SISTEMAS E INFORMÁTICA**

BUCARAMANGA

2025

DESARROLLO DE UNA APLICACIÓN WEB PARA LA GESTIÓN INTEGRADA DE INVENTARIO, VENTAS Y CLIENTES EN PEQUEÑOS NEGOCIOS

SANTIAGO ENRIQUE VILLABONA APONTE

Trabajo de grado presentado para optar al título de
INGENIERO DE SISTEMAS E INFORMÁTICA

Director:

ING. DANITH PATRICIA SOLORZANO ESCOBAR

UNIVERSIDAD PONTIFICIA BOLIVARIANA
ESCUELA DE INGENIERÍAS
INGENIERÍA DE SISTEMAS E INFORMÁTICA
BUCARAMANGA

2025

DEDICATORIA

Texto de la dedicatoria

Nombre del autor

AGRADECIMIENTOS

Texto de los agradecimientos

CONTENIDO

I. INTRODUCCIÓN	8
II. DELIMITACIÓN DEL PROBLEMA	9
A. Situación problema	9
III. ANTECEDENTES	11
IV. JUSTIFICACIÓN	14
V. OBJETIVOS	20
A. Objetivo general	20
B. Objetivos específicos	20
VI. MARCO TEÓRICO	21
A. Marco Conceptual.....	21
1) Arquitectura de Software	21
2) API REST (Representational State Transfer)	22
3) Gestión de Inventarios.....	23
4) Punto de Venta (POS - Point of Sale)	23
5) Base de Datos Relacional	24
B. Marco Tecnológico.....	24
1) Tecnologías Principales.....	25
2) Herramientas de Apoyo y Bibliotecas Complementarias	27
3) Justificación de la Selección Tecnológica.....	28
VII. METODOLOGÍA	30
A. Enfoque Metodológico.....	30
1) Principios de Desarrollo	30
B. Fases del Proyecto.....	31
1) Fase 1: Análisis, Diseño y Configuración (Semanas 1-2).....	31

2) Fase 2: Backend Core (Semanas 3-5)	32
3) Fase 3: Frontend Core (Semanas 6-8).....	32
4) Fase 4: Integración, Pruebas y Refinamiento (Semanas 9-10).....	33
5) Fase 5: Validación y Despliegue (Semana 11).....	34
6) Fase 6: Documentación Final (Semana 12).....	34
C. Distribución Semanal Recomendada	35
D. Hitos de Control	35
VIII. PRESUPUESTO	36
A. Notas sobre el Presupuesto	36
IX. RESULTADOS	38
A. Requerimientos del Sistema	38
B. Requerimientos Funcionales.....	38
C. Requerimientos No Funcionales.....	38
D. Ejemplo de una lista	38
E. Ejemplo de una figura	38
X. CONCLUSIONES.....	39
BIBLIOGRAFÍA	40

LISTA DE TABLAS

TABLA I	Cronograma de Ejecución del Proyecto	35
TABLA II	Distribución Semanal de Actividades	36
TABLA III	Presupuesto Estimado del Proyecto	37

GLOSARIO

SIGLA	SIGNIFICADO
ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object
ERP	Enterprise Resource Planning
HTTP	HyperText Transfer Protocol
IA	Inteligencia Artificial
IP	Internet Protocol
IVA	Impuesto al Valor Agregado
JWT	JSON Web Token
MVC	Model-View-Controller
MVP	Minimum Viable Product
ORM	Object-Relational Mapping
POS	Point of Sale
REST	Representational State Transfer
SKU	Stock Keeping Unit
SQL	Structured Query Language
SSG	Static Site Generation
SSR	Server-Side Rendering
UI	User Interface
UX	User Experience

I. INTRODUCCIÓN

El presente proyecto de grado tiene como objetivo principal el desarrollo de una aplicación web que permita a pequeños negocios colombianos gestionar de manera integrada sus procesos de inventario, ventas y clientes. En un contexto empresarial donde las micro y pequeñas empresas enfrentan dificultades para adoptar soluciones tecnológicas debido a costos elevados y complejidad de uso, surge la necesidad de crear herramientas accesibles y funcionales que faciliten la digitalización de sus operaciones.

La propuesta se fundamenta en el diseño y desarrollo de un sistema que permita llevar un seguimiento detallado del inventario de productos, registrar transacciones de venta en tiempo real, gestionar información de clientes y generar reportes que faciliten la toma de decisiones. Con esta herramienta, se busca mejorar la eficiencia operativa, reducir errores en el manejo de información y proporcionar mayor visibilidad sobre el estado del negocio.

El documento se estructura en varias secciones fundamentales. En primer lugar, se presenta la situación problema que motiva el desarrollo del proyecto, identificando las necesidades específicas de los pequeños negocios y estableciendo la pregunta de investigación. Posteriormente, se definen el objetivo general y los objetivos específicos que guiarán el desarrollo del sistema.

La sección de antecedentes presenta una revisión de soluciones similares existentes en el mercado, mientras que la justificación expone la relevancia y pertinencia del proyecto. El marco referencial establece las bases conceptuales y tecnológicas necesarias para abordar el problema, incluyendo conceptos de arquitectura de software, desarrollo web y gestión de inventarios.

La metodología describe el enfoque iterativo e incremental que se seguirá para el desarrollo del sistema, dividido en fases claramente definidas que abarcan desde el análisis y diseño hasta la validación con usuarios reales. Finalmente, se presenta la arquitectura del sistema, detallando la estructura modular por capas y los módulos funcionales que compondrán la aplicación.

Este proyecto representa una oportunidad para aplicar los conocimientos adquiridos durante la formación académica en Ingeniería de Sistemas, demostrando competencias en análisis de requisitos, diseño de software, implementación de tecnologías modernas y validación de soluciones informáticas en contextos reales.

II. DELIMITACIÓN DEL PROBLEMA

A. *Situación problema*

En el contexto de pequeños negocios colombianos, especialmente dentro del sector de las micro y pequeñas empresas, persiste una brecha significativa en cuanto a la digitalización y centralización de procesos administrativos básicos. Aunque la adopción de herramientas tecnológicas ha aumentado en los últimos años, gran parte de estos negocios continúa gestionando sus operaciones mediante hojas de cálculo, cuadernos físicos o aplicaciones aisladas que no se integran entre sí. Esta fragmentación limita la trazabilidad de la información, genera inconsistencias en los datos y dificulta la toma de decisiones informada.

La falta de integración entre los procesos fundamentales, como el manejo de inventarios, el registro de ventas, la gestión de clientes y la generación de reportes, ocasiona errores recurrentes, pérdida de información, duplicación de registros y poca visibilidad sobre el estado real del negocio. Esta situación afecta directamente la eficiencia operativa, el control sobre los recursos y la capacidad de crecimiento sostenible de los pequeños negocios.

Los propietarios de estos establecimientos a menudo enfrentan desafíos específicos en su día a día. El control de inventario deficiente se manifiesta cuando no saben con certeza qué productos tienen en stock, cuáles están por agotarse o cuál es el valor total de su inventario, generando compras innecesarias o faltantes de productos importantes. Las ventas sin trazabilidad representan otro problema crítico, ya que las transacciones se registran manualmente o de forma desorganizada, dificultando la identificación de productos más vendidos, períodos de mayor actividad o el seguimiento de pagos pendientes.

La gestión de clientes limitada implica que no existe un registro centralizado de clientes frecuentes, su historial de compras o información de contacto, perdiendo oportunidades de fidelización y análisis de comportamiento. Finalmente, los reportes inexistentes o manuales significan que la generación de reportes para evaluar el desempeño del negocio requiere horas de trabajo manual y consolidación de datos dispersos, cuando no se realiza directamente.

Aunque existen soluciones comerciales en el mercado, como sistemas ERP o plataformas de administración empresarial, muchas de ellas presentan costos elevados, funcionalidades innecesarias para negocios en etapa inicial o una curva de aprendizaje compleja. Además, requieren inversiones continuas en licencias y actualizaciones que no todos los pequeños empresarios pueden

asumir, lo que limita el acceso a herramientas tecnológicas que podrían mejorar significativamente su gestión.

Por otro lado, las soluciones gratuitas disponibles suelen estar orientadas a mercados internacionales, con interfaces en otros idiomas, procesos no adaptados al contexto colombiano y limitaciones funcionales significativas en sus versiones gratuitas. Esto genera frustración y abandono de las herramientas, retornando a los métodos manuales tradicionales.

Ante este panorama, se identifica la oportunidad de desarrollar una aplicación web que permita a pequeños negocios gestionar sus procesos operativos y administrativos de forma centralizada y accesible, demostrando la aplicación práctica de conocimientos en arquitectura de software, desarrollo full-stack y buenas prácticas de ingeniería. Con base en ello, surge la siguiente pregunta de investigación:

¿Cómo desarrollar una aplicación web que permita a pequeños negocios gestionar de manera integrada sus procesos de inventario, ventas, clientes y reportes, mediante una arquitectura modular por capas que garantice organización, mantenibilidad y escalabilidad?

III. ANTECEDENTES

El manejo eficiente de recursos en la gestión de inventarios y ventas constituye una problemática recurrente y significativa que enfrentan las pequeñas empresas, particularmente aquellas compañías de escasos recursos económicos o que se encuentran en las etapas iniciales de sus operaciones comerciales. Estas organizaciones frecuentemente carecen de acceso a soluciones tecnológicas adecuadas que se ajusten a sus necesidades específicas y limitaciones presupuestarias, viéndose obligadas a conformarse con herramientas rudimentarias de funcionalidad limitada o a depender de métodos manuales que consumen tiempo valioso y son propensos a errores humanos. Esta situación genera una desventaja competitiva significativa frente a empresas más establecidas que cuentan con sistemas robustos de gestión empresarial.

En el mercado actual existen diversas plataformas comerciales que ofrecen soluciones tecnológicas dirigidas a esta problemática operativa, muchas de ellas desarrolladas y comercializadas por grandes empresas de software con décadas de experiencia en el sector empresarial. Entre las soluciones más reconocidas y ampliamente adoptadas en el contexto latinoamericano se encuentran sistemas como Odoo Inventory, Alegra y Siigo, cada uno con características distintivas, fortalezas particulares y limitaciones específicas que es importante analizar para contextualizar adecuadamente la propuesta del presente proyecto.

Odoo Inventory se posiciona como uno de los principales servicios de manejo de inventario a nivel mundial, definido por sus creadores como una plataforma única e integral necesaria para administrar diversos aspectos del negocio. Esta solución cuenta con aplicaciones modulares integradas entre sí de manera coherente y sencilla, siendo utilizada actualmente por millones de usuarios en diferentes países e industrias. Odoo ayuda a las empresas a administrar su inventario de productos o activos operativos sin obstáculos significativos, proporcionando funcionalidades avanzadas como órdenes de entrega automatizadas, portales dedicados para clientes, sistemas de alertas personalizables según las necesidades específicas del negocio, y un planificador inteligente que sugiere acciones basándose en patrones históricos de consumo y venta.

Sin embargo, a pesar de sus capacidades técnicas impresionantes y su interfaz relativamente intuitiva, Odoo Inventory presenta una limitación fundamental para el contexto de pequeños negocios con recursos limitados. Este sistema opera bajo un modelo de negocio de pago, ofreciendo únicamente una versión de prueba temporal con funcionalidades completas, pero una vez finaliza-

do el período de evaluación, comienza a facturar mensualmente por el uso continuado del servicio. Este modelo de suscripción recurrente, aunque común en la industria del software como servicio, representa una barrera de entrada significativa para microempresas y pequeños negocios que operan con márgenes ajustados y deben priorizar cuidadosamente cada gasto operativo. En contraste, el sistema propuesto en el presente proyecto adopta un enfoque académico y de demostración de conceptos arquitectónicos, sin las restricciones comerciales asociadas con licenciamiento y pagos recurrentes.

Alegra representa otra plataforma prominente en el ecosistema colombiano de gestión empresarial, destacándose particularmente por su enfoque en facturación electrónica y gestión contable integrada. Desarrollada específicamente con las necesidades del mercado colombiano en mente, Alegra ofrece herramientas especializadas para el manejo de inventarios, facturación electrónica que cumple rigurosamente con los requisitos establecidos por la Dirección de Impuestos y Aduanas Nacionales de Colombia, y control detallado de ventas con capacidades de análisis y reportería. Su adaptación al contexto regulatorio colombiano constituye una ventaja significativa, ya que incorpora nativamente el cálculo correcto de impuestos locales, los formatos requeridos para documentos tributarios, y la integración con los sistemas de la DIAN para el envío de facturas electrónicas.

No obstante, similar a Odoo, los planes de pago de Alegra pueden resultar económicamente elevados para pequeños negocios que apenas están iniciando sus operaciones o que se encuentran en etapas de crecimiento temprano con ingresos aún modestos. Adicionalmente, muchas de las funcionalidades más avanzadas y valiosas del sistema se encuentran limitadas o completamente inaccesibles en las versiones gratuitas, obligando a los usuarios a actualizar a planes de pago para acceder a capacidades que podrían considerar esenciales para su operación diaria. Esta estrategia de monetización mediante limitación de funcionalidades, aunque comprensible desde una perspectiva de negocio, crea frustración en usuarios que descubren tardíamente que las capacidades que realmente necesitan requieren inversión adicional.

Siigo constituye otro sistema de gestión empresarial colombiano ampliamente utilizado, especialmente reconocido y valorado en el ámbito contable y de facturación electrónica por su robustez y cumplimiento normativo. Esta plataforma ofrece módulos integrados de inventario y ventas que funcionan de manera coordinada con sus capacidades contables centrales, permitiendo mantener sincronizados los registros de movimientos de mercancía con los asientos contables

correspondientes. La fortaleza principal de Siigo radica en su profundidad contable y su capacidad para generar reportes financieros detallados que satisfacen requisitos tanto internos de gestión como externos de cumplimiento tributario.

Sin embargo, la principal limitación de Siigo para el segmento de pequeños negocios reside en la complejidad inherente de uso para usuarios que carecen de formación o experiencia contable previa. El sistema asume frecuentemente un nivel de conocimiento contable que muchos pequeños empresarios simplemente no poseen, resultando en una curva de aprendizaje pronunciada que puede desalentar su adopción o generar uso inadecuado de sus funcionalidades. Adicionalmente, similar a las plataformas anteriormente mencionadas, Siigo opera bajo un modelo de costos de licenciamiento mensual o anual que debe ser cuidadosamente evaluado en el contexto de las finanzas de un pequeño negocio.

Más allá de las soluciones comerciales predominantes en el mercado, se han implementado diversos estudios académicos y proyectos de investigación relacionados con sistemas de gestión para pequeñas empresas. Estas investigaciones abordan temas como arquitecturas de software óptimas para aplicaciones de gestión empresarial, patrones de diseño apropiados para sistemas transaccionales, estrategias de optimización de bases de datos para consultas de reportería, y metodologías de desarrollo ágil adaptadas a proyectos de gestión. Proyectos de grado anteriores en instituciones universitarias han explorado soluciones similares enfocadas en sectores específicos, desarrollando sistemas especializados para inventarios en contextos particulares como farmacias, ferreterías, restaurantes o tiendas de ropa.

La revisión de esta literatura académica y proyectos previos proporciona valiosas lecciones aprendidas sobre qué enfoques arquitectónicos han demostrado ser más exitosos en la práctica, qué patrones de diseño facilitan el mantenimiento y la extensibilidad del código, qué tecnologías han mostrado mejor desempeño en escenarios reales de uso, y qué dificultades comunes enfrentan los desarrolladores al construir este tipo de sistemas. Esta fundamentación en trabajos previos permite al presente proyecto construir sobre conocimiento consolidado en lugar de partir desde cero, evitando errores ya identificados y adoptando mejores prácticas ya validadas en contextos similares.

IV. JUSTIFICACIÓN

Los pequeños negocios que gestionan operaciones de inventario, ventas y relaciones con clientes enfrentan cotidianamente múltiples desafíos operativos significativos que impactan negativamente tanto su eficiencia diaria como sus perspectivas de crecimiento sostenible a mediano y largo plazo. La acumulación y dispersión de información crítica del negocio en diferentes medios heterogéneos, tales como cuadernos físicos manuscritos, hojas de cálculo electrónicas desconectadas entre sí, notas adhesivas efímeras, o registros mentales no documentados, genera consecuencias problemáticas de desorganización generalizada, pérdida frecuente de datos esenciales para la operación, y dificultad sustancial para tomar decisiones empresariales informadas basadas en información confiable y actualizada sobre el verdadero estado del negocio.

La gestión verdaderamente eficiente y efectiva de estos recursos operativos críticos no siempre se logra alcanzar exitosamente debido a varios factores limitantes que actúan simultáneamente creando barreras de entrada significativas. Entre estos factores destacan principalmente el costo económico directo que implica pagar suscripciones mensuales o anuales recurrentes a las diversas plataformas comerciales que existen en el mercado para este propósito específico. La complejidad técnica de uso de sistemas empresariales robustos diseñados para organizaciones grandes resulta frecuentemente abrumadora y excede significativamente las necesidades reales y capacidades técnicas de un pequeño negocio que requiere soluciones más directas y accesibles. Adicionalmente, la falta de adaptación adecuada de soluciones internacionales desarrolladas para mercados norteamericanos o europeos al contexto local colombiano, incluyendo aspectos regulatorios específicos, prácticas comerciales locales, y consideraciones culturales particulares, genera fricciones adicionales en su adopción efectiva.

Por estas razones fundamentales ampliamente documentadas en la problemática planteada, se propone la creación y desarrollo de una aplicación web específicamente diseñada para abordar esta problemática identificada de manera directa y efectiva. Esta solución propuesta ofrece ventajas tangibles en múltiples dimensiones que benefician tanto los aspectos operativos inmediatos del negocio como los objetivos académicos de formación profesional del estudiante desarrollador.

Desde la perspectiva de beneficios operativos concretos para el negocio usuario final, la centralización integral de información constituye quizás la ventaja más inmediata y apreciable. Toda la información crítica del negocio, incluyendo el catálogo completo de productos con sus carac-

terísticas y precios, el historial detallado de todas las ventas realizadas con sus montos y fechas, y los registros completos de clientes con su información de contacto e historial de compras, se encuentra almacenada y accesible desde un único punto centralizado. Esta centralización elimina completamente la dispersión problemática de datos en múltiples ubicaciones desconectadas y facilita enormemente la consulta rápida de cualquier información necesaria en el momento preciso en que se requiere para tomar una decisión o atender una consulta.

La trazabilidad completa y exhaustiva de todas las operaciones representa otro beneficio operativo de gran valor para la gestión efectiva del negocio. Cada operación individual realizada en el sistema, sin importar si se trata de una entrada de nuevo inventario, una venta completada, un ajuste de precios, o una modificación de información de cliente, queda registrada permanentemente con su fecha exacta, hora precisa, y detalles completos que permiten reconstruir exactamente qué sucedió en cualquier momento del pasado. Esta capacidad de auditoría completa permite rastrear retrospectivamente el historial completo de movimientos de inventario para identificar patrones de consumo, analizar todas las ventas realizadas para detectar tendencias temporales o estacionales, y examinar la evolución de la relación comercial con cada cliente individual a lo largo del tiempo.

La reducción significativa de errores humanos mediante automatización de cálculos repetitivos y propensos a equivocaciones manuales constituye otra ventaja operativa crucial. La automatización completa de cálculos numéricos como la suma de totales de venta, el cálculo correcto del Impuesto al Valor Agregado según las tasas vigentes, y el control preciso de niveles de stock considerando entradas y salidas, minimiza drásticamente los errores humanos que inevitablemente ocurren cuando estas operaciones se realizan manualmente, especialmente bajo presión de tiempo o fatiga durante jornadas laborales extensas. Esta automatización garantiza precisión consistente en operaciones críticas independientemente del volumen de transacciones o la complejidad de los cálculos involucrados.

La accesibilidad universal desde prácticamente cualquier dispositivo conectado representa una ventaja técnica importante en el contexto operativo moderno. Al tratarse fundamentalmente de una aplicación web estándar basada en tecnologías universales, el sistema puede accederse exitosamente desde cualquier dispositivo que cuente con un navegador web moderno, incluyendo computadores de escritorio tradicionales, laptops portátiles, tablets de diversos tamaños, e incluso teléfonos inteligentes cuando sea necesario consultar información urgentemente fuera de las insta-

laciones físicas del negocio. Esta accesibilidad multiplataforma se logra sin requerir instalaciones complejas de software especializado, configuraciones técnicas avanzadas, o descargas de aplicaciones nativas que consumen espacio de almacenamiento limitado en dispositivos móviles.

La generación instantánea y automatizada de reportes y métricas empresariales que anteriormente requerían invertir horas valiosas de trabajo manual tedioso representa quizás la ventaja más transformadora para la toma de decisiones estratégicas. Reportes que tradicionalmente demandaban dedicar una tarde completa a consolidar manualmente datos dispersos en múltiples documentos ahora pueden generarse instantáneamente con un simple clic, presentando la información de manera visual y comprensible mediante gráficas, tablas y resúmenes ejecutivos que facilitan identificar rápidamente oportunidades, problemas emergentes, o tendencias significativas que requieren atención o acción.

Desde la perspectiva de beneficios académicos para el desarrollo profesional del estudiante, el proyecto representa una oportunidad invaluable de aplicación práctica e integrada de conocimientos teóricos adquiridos durante toda la formación en Ingeniería de Sistemas. El diseño e implementación de arquitectura de software modular por capas permite aplicar directamente principios arquitectónicos fundamentales como la separación clara de responsabilidades entre diferentes componentes del sistema, el bajo acoplamiento que permite modificar componentes independientemente, y la alta cohesión que agrupa funcionalidades relacionadas. Esta experiencia práctica de diseño arquitectónico trasciende el conocimiento teórico de libros de texto y proporciona comprensión profunda de por qué estos principios importan genuinamente en proyectos reales de software.

La experiencia de desarrollo full-stack integral, abarcando tanto el backend implementado con NestJS y PostgreSQL como el frontend construido con Next.js y React, proporciona una visión holística y completa del desarrollo de aplicaciones web modernas. Esta experiencia de trabajar simultáneamente en ambos extremos del stack tecnológico, integrando ambas capas mediante una API REST bien diseñada y documentada, desarrolla habilidades versátiles altamente valoradas en la industria actual donde la demanda de desarrolladores capaces de moverse fluidamente entre frontend y backend supera significativamente la oferta disponible de profesionales con estas capacidades.

La gestión efectiva de proyectos de software mediante la aplicación de metodologías iterativas e incrementales adaptadas al contexto específico de trabajo individual, la gestión organizada de

tareas mediante tableros Kanban visuales que proporcionan claridad sobre el progreso, y el control riguroso de versiones con Git que permite rastrear cambios y experimentar con confianza, desarrolla competencias críticas de ingeniería de software que trascienden cualquier tecnología o lenguaje particular y permanecen relevantes a lo largo de toda una carrera profesional.

El diseño exhaustivo de modelos relacionales de bases de datos, aplicando rigurosamente principios de normalización para eliminar redundancias problemáticas, diseñando consultas optimizadas que ejecutan eficientemente incluso con grandes volúmenes de datos, y gestionando apropiadamente transacciones que garantizan consistencia de datos incluso ante fallos, proporciona experiencia práctica invaluable en un área fundamental que sustenta prácticamente cualquier aplicación empresarial moderna de cierta complejidad.

La ingeniería de requisitos ejecutada desde las primeras fases del proyecto, identificando sistemáticamente necesidades reales de usuarios objetivo, definiendo claramente requisitos funcionales que especifican qué debe hacer el sistema y requisitos no funcionales que establecen bajo qué condiciones debe operar, y documentando casos de uso que describen interacciones específicas entre usuarios y sistema, desarrolla habilidades críticas de análisis que determinan frecuentemente el éxito o fracaso de proyectos de software independientemente de la calidad técnica de la implementación.

Finalmente, la implementación sistemática de pruebas de software en múltiples niveles, incluyendo pruebas unitarias que verifican componentes individuales aisladamente, pruebas de integración que validan interacciones entre componentes, y validación con usuarios reales que evalúa usabilidad y adecuación a necesidades reales, proporciona experiencia práctica en aseguramiento de calidad que distingue software amateur de software profesional apto para uso productivo real.

Desde la perspectiva de impacto social potencial en el contexto de pequeños negocios colombianos, la implementación exitosa de este sistema tiene ramificaciones positivas que trascienden el beneficio individual de cualquier negocio particular. La democratización efectiva de acceso a tecnología de gestión empresarial, proporcionando herramientas profesionales de calidad a negocios que carecen de recursos financieros para costear soluciones comerciales con licenciamiento costoso, contribuye a nivelar parcialmente el campo de juego competitivo entre empresas de diferentes tamaños y recursos.

La mejora tangible de competitividad que experimentan pequeños negocios cuando adquie-

ren mejor control y visibilidad sobre su información operativa les permite competir más eficazmente en mercados donde tradicionalmente empresas más grandes con mejores sistemas dominaban gracias a su superior capacidad de análisis y respuesta basada en datos confiables y oportunos.

La facilitación de procesos de formalización de negocios que operan informalmente, mediante el registro ordenado y sistemático de operaciones que anteriormente no se documentaban adecuadamente, contribuye positivamente a la economía formal, mejora el cumplimiento tributario, y facilita el acceso eventual a créditos y otros servicios financieros que requieren demostración de flujos de ingresos documentados.

Finalmente, la generación de conocimiento académico mediante este proyecto puede servir como fundamento sólido para futuros trabajos de investigación que extiendan sus capacidades, estudien su efectividad en diferentes contextos, o exploren problemáticas relacionadas, contribuyendo así al cuerpo de conocimiento académico en el área de sistemas de información para pequeñas empresas.

Desde el punto de vista de viabilidad técnica del proyecto propuesto, múltiples factores confluyen para garantizar que su ejecución exitosa es realista y alcanzable dentro de las restricciones temporales y de recursos disponibles. El uso exclusivo de tecnologías de código abierto ampliamente documentadas elimina costos de licenciamiento y garantiza acceso a recursos educativos abundantes. La selección de un stack tecnológico moderno respaldado por comunidades grandes y activas asegura que cualquier problema técnico encontrado probablemente ya ha sido resuelto y documentado públicamente. La posibilidad real de despliegue completamente gratuito en plataformas como Vercel para frontend y Railway para backend permite demostrar el sistema funcionando en producción sin incurrir en costos de infraestructura que limitarían proyectos académicos. Finalmente, el alcance cuidadosamente delimitado a funcionalidades esenciales, evitando conscientemente complejidad innecesaria que no contribuye directamente a los objetivos académicos establecidos, garantiza que el proyecto permanezca manejable dentro del tiempo disponible.

En resumen comprehensivo, este proyecto representa una oportunidad única y valiosa para abordar una problemática real y significativa que enfrentan cotidianamente los pequeños negocios colombianos, mientras simultáneamente se demuestra la aplicación práctica e integrada de conocimientos técnicos adquiridos durante toda la formación académica en Ingeniería de Sistemas. Esta doble contribución, tanto al desarrollo profesional del estudiante mediante experiencia práctica pro-

funda como al mejoramiento de la gestión empresarial en el sector objetivo de pequeños negocios, justifica plenamente el esfuerzo y los recursos que se invertirán en la ejecución exitosa del proyecto propuesto.

V. OBJETIVOS

A. *Objetivo general*

Desarrollar una aplicación web que integre la gestión de inventario, ventas, clientes y reportes, demostrando la aplicación de arquitectura modular por capas, patrones de diseño y tecnologías modernas de desarrollo full-stack, en el contexto de la administración de pequeños negocios.

B. *Objetivos específicos*

- Definir la estructura del sistema mediante capas de presentación, lógica de negocio y acceso a datos, estableciendo los módulos funcionales (inventario, ventas, clientes, reportes), sus responsabilidades y mecanismos de comunicación, garantizando separación de responsabilidades y escalabilidad.
- Desarrollar la capa de servidor mediante una arquitectura RESTful que implemente la lógica de negocio de los módulos de inventario, ventas, clientes y reportes, garantizando la persistencia de datos, validación de reglas de negocio y exposición de servicios mediante endpoints documentados.
- Construir la capa de presentación mediante componentes reutilizables que permitan la interacción del usuario con los módulos del sistema, implementando formularios, tablas, filtros y visualizaciones de datos, priorizando la usabilidad y accesibilidad desde diferentes dispositivos.
- Verificar el correcto funcionamiento de los módulos implementados mediante pruebas de integración, pruebas de casos de uso completos y evaluación con usuarios potenciales, documentando hallazgos y realizando ajustes necesarios para garantizar la calidad del sistema.

VI. MARCO TEÓRICO

Para el desarrollo del proyecto se realizó una investigación exhaustiva sobre los fundamentos teóricos y tecnológicos necesarios para construir un sistema de gestión integrado robusto y escalable. Esta sección se divide en dos componentes fundamentales: el marco conceptual, que establece las bases teóricas sobre las cuales se construye el sistema, y el marco tecnológico, que describe detalladamente las herramientas, frameworks y tecnologías seleccionadas para la implementación del proyecto.

A. *Marco Conceptual*

1) *Arquitectura de Software:* La arquitectura de software constituye el pilar fundamental sobre el cual se construye cualquier sistema informático moderno. Se refiere a la estructura organizacional de alto nivel de un sistema, que comprende sus componentes principales, las relaciones establecidas entre ellos, y los principios rectores que guían tanto su diseño inicial como su evolución a lo largo del tiempo. Una arquitectura bien definida y documentada no solo facilita significativamente el mantenimiento del sistema a largo plazo, sino que también potencia su escalabilidad, mejora sustancialmente su comprensión por parte de los desarrolladores actuales y futuros, y permite realizar modificaciones y extensiones de manera más controlada y predecible.

La arquitectura por capas representa uno de los patrones arquitectónicos más consolidados y ampliamente adoptados en la industria del software. Este patrón organiza metódicamente el sistema en capas horizontales superpuestas, donde cada capa asume una responsabilidad específica y bien delimitada dentro del ecosistema del sistema. Una característica fundamental de este patrón es que cada capa se comunica exclusivamente con las capas inmediatamente adyacentes, estableciendo así un flujo de comunicación controlado y predecible. Esta organización jerárquica proporciona múltiples ventajas arquitectónicas: en primer lugar, promueve una clara separación de responsabilidades, lo que significa que cada capa se enfoca en resolver un conjunto específico de problemas sin interferir con las demás. En segundo lugar, facilita enormemente el mantenimiento del código, ya que los cambios en una capa específica tienen un impacto limitado y controlado en el resto del sistema. Adicionalmente, esta arquitectura ofrece la posibilidad de reemplazar la implementación completa de una capa sin necesidad de modificar las capas restantes, siempre que se respete la interfaz de comunicación establecida. Finalmente, contribuye a una mejor organización del código fuente, haciendo que la estructura del proyecto sea más intuitiva y navegable.

En el contexto específico del presente proyecto, se implementa una arquitectura de tres capas claramente diferenciadas. La primera capa, denominada capa de presentación, es responsable de toda la interfaz de usuario y está implementada mediante el framework Next.js en el lado del cliente. Esta capa gestiona la visualización de información, la captura de entradas del usuario, y la presentación de datos de manera comprensible y atractiva. La segunda capa, conocida como capa de lógica de negocio, contiene todo el procesamiento de datos y las reglas de negocio que definen cómo opera el sistema. Implementada con NestJS en el servidor, esta capa orquesta las operaciones complejas, valida datos, aplica reglas empresariales y coordina las interacciones entre diferentes módulos del sistema. La tercera capa, llamada capa de acceso a datos, se encarga exclusivamente de la persistencia y recuperación de información desde y hacia la base de datos PostgreSQL, abstrayendo los detalles de almacenamiento del resto del sistema.

2) API REST (*Representational State Transfer*): REST constituye un estilo arquitectónico fundamental para sistemas distribuidos, especialmente diseñado para servicios web modernos. Una API REST aprovecha los métodos HTTP estándar, específicamente GET para recuperar recursos, POST para crear nuevos recursos, PUT para actualizar recursos existentes, y DELETE para eliminar recursos, con el fin de realizar operaciones sobre recursos identificados de manera única mediante URLs estructuradas y semánticas.

Las características principales de REST definen su filosofía operativa. El principio de ausencia de estado, conocido como stateless, establece que cada petición HTTP debe contener toda la información necesaria para ser procesada de manera independiente, sin depender de peticiones previas o del contexto almacenado en el servidor. La separación clara entre cliente y servidor permite que ambos componentes evolucionen de manera independiente, facilitando la escalabilidad y el mantenimiento. La capacidad de cachear respuestas proporciona mejoras significativas en el rendimiento, ya que las respuestas pueden ser almacenadas temporalmente para reducir la carga del servidor y los tiempos de respuesta. Finalmente, la interfaz uniforme garantiza el uso consistente de métodos HTTP y URIs, creando un contrato predecible entre cliente y servidor.

En el proyecto actual, la API REST constituye el puente de comunicación fundamental entre el frontend desarrollado en Next.js y el backend implementado con NestJS. Esta API expone endpoints claramente definidos que permiten realizar todas las operaciones necesarias para gestionar inventario, procesar ventas, administrar clientes y generar reportes. La documentación automáti-

ca de estos endpoints mediante Swagger facilita su comprensión y uso tanto durante el desarrollo como en fases posteriores de mantenimiento.

3) *Gestión de Inventarios:*La gestión de inventarios representa un proceso empresarial crítico que involucra la supervisión continua y el control meticuloso de las existencias de productos dentro de una organización. Este proceso abarca múltiples dimensiones operativas que deben funcionar en conjunto de manera armoniosa. El control de stock implica el registro detallado y actualizado de todas las entradas y salidas de productos, manteniendo en todo momento una visión precisa de las cantidades disponibles. La valorización del inventario requiere el cálculo constante del valor total del stock, considerando tanto los precios de costo como los precios de venta, lo que proporciona información financiera crucial para la toma de decisiones. La trazabilidad garantiza la capacidad de seguir el historial completo de movimientos de cada producto individual, desde su ingreso al inventario hasta su eventual venta o ajuste. Las alertas automáticas proporcionan notificaciones oportunas cuando los niveles de stock de ciertos productos caen por debajo de umbrales mínimos predefinidos, permitiendo una reposición proactiva.

Para pequeños negocios, una gestión efectiva de inventarios desencadena múltiples beneficios operativos y financieros. Permite evitar los costosos quiebres de stock que resultan en ventas perdidas y clientes insatisfechos. Reduce el capital inmovilizado en inventario excesivo, liberando recursos financieros para otras necesidades del negocio. Facilita la toma de decisiones informadas sobre compras futuras, basándose en patrones de consumo reales. Finalmente, ayuda a identificar los productos de mayor rotación, permitiendo optimizar la mezcla de productos ofrecidos.

4) *Punto de Venta (POS - Point of Sale):*Un sistema punto de venta representa la interfaz tecnológica que permite registrar transacciones comerciales en tiempo real, constituyendo el punto de contacto crítico entre el negocio y sus clientes durante el proceso de compra. Sus componentes típicos operan en conjunto para crear una experiencia de venta fluida. La búsqueda rápida de productos permite localizar items específicos en segundos, acelerando significativamente el proceso de venta. El carrito de compra digital facilita agregar, modificar cantidades y eliminar items de manera intuitiva antes de finalizar la transacción. El cálculo automático de totales e impuestos elimina errores humanos y garantiza precisión en los montos cobrados. El registro detallado de métodos de pago proporciona trazabilidad financiera completa. La generación automática de comprobantes ofrece documentación instantánea de cada transacción.

En el contexto específico de pequeños negocios, el sistema POS debe cumplir con requisitos particulares adaptados a sus necesidades operativas. Debe ser extremadamente rápido, permitiendo completar una venta en cuestión de segundos para mantener la fluidez del servicio. Necesita ser intuitivo, con una curva de aprendizaje mínima que permita a cualquier empleado operarlo eficientemente tras una capacitación breve. Debe ser absolutamente confiable, ejecutando la reducción automática de stock sin errores ni inconsistencias que puedan generar problemas posteriores de inventario.

5) Base de Datos Relacional: Las bases de datos relacionales representan un sistema de organización de información altamente estructurado que almacena datos en tablas interconectadas mediante relaciones definidas por claves primarias y claves foráneas. Estas bases de datos utilizan SQL (Structured Query Language) como lenguaje estándar para realizar consultas complejas y manipular datos de manera eficiente y controlada.

Las ventajas que aportan las bases de datos relacionales al presente proyecto son múltiples y fundamentales. La integridad referencial garantiza automáticamente la consistencia entre datos relacionados, evitando inconsistencias como registros huérfanos o referencias a entidades inexistentes. Las transacciones ACID proporcionan garantías críticas de atomicidad (las operaciones se completan totalmente o no se realizan), consistencia (el sistema siempre mantiene un estado válido), aislamiento (las transacciones concurrentes no interfieren entre sí) y durabilidad (los cambios confirmados persisten incluso ante fallos del sistema). La capacidad de realizar consultas complejas permite operaciones sofisticadas como joins entre múltiples tablas, agregaciones estadísticas y filtros avanzados que facilitan la generación de reportes detallados. La normalización de datos reduce significativamente la redundancia de información, optimizando el uso de espacio de almacenamiento y simplificando las actualizaciones de datos.

B. Marco Tecnológico

El desarrollo de la aplicación se fundamenta en un conjunto cuidadosamente seleccionado de herramientas tecnológicas modernas, escalables y de código abierto. Esta selección estratégica busca garantizar la construcción de un sistema robusto, fácilmente mantenible y capaz de adaptarse a las exigencias cambiantes del desarrollo de software contemporáneo. Cada tecnología ha sido evaluada no solo por sus capacidades técnicas intrínsecas, sino también por su madurez en el mercado, el tamaño y actividad de su comunidad de soporte, la calidad de su documentación, y su

compatibilidad con las demás herramientas del stack tecnológico.

1) *Tecnologías Principales:*Next.js constituye el framework fundamental para el desarrollo del frontend de la aplicación. Construido sobre la base sólida de React, Next.js proporciona capacidades avanzadas que van mucho más allá de las ofrecidas por React básico. Su soporte nativo para renderizado del lado del servidor permite que las páginas se generen dinámicamente en el servidor antes de ser enviadas al cliente, mejorando significativamente tanto el rendimiento percibido como el SEO de la aplicación. La generación estática de sitios ofrece la posibilidad de pre-renderizar páginas en tiempo de compilación, obteniendo el máximo rendimiento posible para contenido que no cambia frecuentemente. El sistema de enrutamiento integrado basado en el sistema de archivos simplifica dramáticamente la creación de rutas, eliminando configuraciones complejas. La optimización automática de recursos como imágenes, fuentes y código JavaScript reduce automáticamente el tamaño de los archivos transferidos y mejora los tiempos de carga. El App Router moderno proporciona una estructura de enrutamiento intuitiva y poderosa. Los Server Components permiten ejecutar código React directamente en el servidor, reduciendo el JavaScript enviado al cliente. El soporte nativo para TypeScript garantiza tipado estático en todo el frontend, detectando errores en tiempo de desarrollo.

NestJS representa el framework seleccionado para la construcción del backend de la aplicación. Este framework progresivo, construido sobre Node.js, permite crear APIs modulares, perfectamente organizadas y altamente escalables. Fundamentado en TypeScript desde su concepción, NestJS adopta principios arquitectónicos sólidos que promueven la separación clara de responsabilidades mediante el uso de controladores para manejar peticiones HTTP, servicios para implementar lógica de negocio, y repositorios para gestionar el acceso a datos. Su arquitectura modular facilita la organización del código en módulos funcionales independientes que pueden desarrollarse, probarse y mantenerse de manera aislada. El sistema de decoradores proporciona una sintaxis limpia y expresiva para definir rutas, aplicar validaciones y gestionar dependencias. La inyección de dependencias integrada facilita enormemente tanto el testing como la reutilización de código a través de diferentes módulos. La documentación automática mediante Swagger genera documentación interactiva de la API sin esfuerzo adicional, permitiendo a los desarrolladores explorar y probar endpoints directamente desde el navegador.

TypeScript actúa como el lenguaje de programación unificador utilizado en todas las ca-

pas del proyecto. Al extender JavaScript con un sistema de tipos estático opcional pero poderoso, TypeScript ofrece herramientas que mejoran drásticamente la mantenibilidad del código, reducen significativamente los errores durante el desarrollo, y permiten construir proyectos más robustos a medida que crecen en complejidad. La detección temprana de errores permite al compilador identificar problemas potenciales antes de que el código se ejecute, ahorrando incontables horas de depuración. El autocompletado inteligente proporcionado por los editores modernos mejora exponencialmente la experiencia de desarrollo, sugiriendo métodos, propiedades y parámetros apropiados. La refactorización segura permite realizar cambios estructurales en el código con confianza, ya que el sistema de tipos garantiza que todas las referencias se actualicen consistentemente. La documentación implícita emerge de las definiciones de tipos, que sirven como una forma de documentación siempre actualizada y verificada automáticamente.

PostgreSQL constituye el sistema de gestión de bases de datos relacional seleccionado para el proyecto. Reconocido ampliamente por su confiabilidad excepcional, su naturaleza de código abierto, y su adopción masiva en la industria, PostgreSQL ofrece la solidez necesaria para aplicaciones empresariales mientras mantiene costos de licenciamiento nulos. Su capacidad para manejar estructuras complejas de datos, su riguroso cumplimiento de estándares SQL, y su rendimiento optimizado lo hacen especialmente adecuado para aplicaciones transaccionales como la plataforma de gestión que se está desarrollando. El cumplimiento completo de ACID garantiza la consistencia absoluta de los datos incluso ante fallos del sistema. Los tipos de datos avanzados incluyen soporte nativo para JSON, arrays, y tipos personalizados que permiten modelar datos complejos eficientemente. El rendimiento optimizado se logra mediante índices eficientes, consultas optimizadas automáticamente, y capacidades avanzadas de paralelización. La extensibilidad permite crear funciones personalizadas, triggers automáticos y procedimientos almacenados que encapsulan lógica compleja.

Prisma ORM representa la capa de abstracción seleccionada para facilitar la interacción entre el código de la aplicación y la base de datos PostgreSQL. Como ORM de nueva generación, Prisma adopta un enfoque declarativo mediante un esquema que define todos los modelos de datos en un formato legible y conciso. Este schema sirve como fuente única de verdad para la estructura de datos, a partir del cual Prisma genera automáticamente un cliente tipado para TypeScript. Las migraciones automáticas generan scripts SQL precisos basados en los cambios detectados en el

schema, simplificando enormemente la evolución de la base de datos. El cliente tipado proporciona autocompletado completo y verificación de tipos en tiempo de compilación para todas las consultas, eliminando prácticamente los errores de consulta en tiempo de ejecución. Prisma Studio ofrece una interfaz visual elegante para explorar y manipular datos durante el desarrollo, facilitando la depuración y el testing.

2) Herramientas de Apoyo y Bibliotecas Complementarias: El ecosistema de herramientas de apoyo complementa las tecnologías principales, proporcionando capacidades esenciales para el desarrollo, despliegue y operación del sistema. Git, combinado con GitHub, constituye el sistema de control de versiones distribuido utilizado para gestionar todo el código fuente del proyecto. Esta combinación permite mantener un historial completo y detallado de todos los cambios realizados, facilita el trabajo colaborativo mediante branches y pull requests, y habilita estrategias de despliegue continuo mediante integración con plataformas de hosting. Trello se emplea como herramienta de gestión de tareas, implementando un tablero Kanban visual que organiza el flujo de trabajo del proyecto en columnas que representan diferentes estados de las tareas, desde pendientes hasta completadas.

Figma sirve como herramienta de diseño centralizada para crear prototipos detallados de interfaces UI/UX, desarrollar wireframes que guían la implementación, y establecer un sistema de diseño consistente que se aplica a través de toda la aplicación. Esta herramienta facilita la colaboración en tiempo real y genera especificaciones precisas que los desarrolladores pueden implementar fielmente. Docker proporciona una plataforma de contenedorización que garantiza entornos de desarrollo replicables y absolutamente consistentes entre diferentes máquinas de desarrollo, simplificando significativamente la configuración inicial del proyecto y facilitando eventualmente el despliegue en producción.

Las plataformas de despliegue seleccionadas aprovechan planes gratuitos generosos que son perfectamente adecuados para proyectos académicos. Vercel, optimizada específicamente para aplicaciones Next.js, permite publicar el frontend de forma gratuita con integración continua automática desde GitHub y distribución global mediante su CDN. Railway o Render proporcionan hosting gratuito para el backend NestJS y la base de datos PostgreSQL, ofreciendo capacidades suficientes para desarrollo y demostración del proyecto.

El frontend se enriquece con bibliotecas especializadas cuidadosamente seleccionadas. shadcn/ui

proporciona una colección extensa de componentes UI pre-construidos y altamente personalizables basados en Tailwind CSS y Radix UI, acelerando significativamente el desarrollo de interfaces. TanStack Query maneja elegantemente el estado del servidor y el caché de datos, simplificando las interacciones con la API y optimizando automáticamente las peticiones HTTP. React Hook Form ofrece gestión eficiente de formularios con validación integrada, reduciendo el código boilerplate necesario. Zod proporciona validación robusta de esquemas TypeScript, garantizando la integridad de datos tanto en el cliente como en el servidor. Recharts facilita la creación de gráficas atractivas y responsivas para la visualización de métricas y reportes.

El backend se complementa con herramientas específicas de NestJS. class-validator proporciona decoradores elegantes para validar DTOs de manera declarativa. class-transformer permite la transformación automática de objetos planos en instancias de clases tipadas. Swagger y OpenAPI generan documentación interactiva automática de todos los endpoints de la API, manteniendo siempre sincronizada la documentación con la implementación real.

Cloudinary se selecciona como servicio de gestión de imágenes en la nube, ofreciendo almacenamiento confiable, optimización automática de imágenes, transformaciones on-the-fly, y un CDN global para servir imágenes rápidamente. Su plan gratuito de 25GB mensuales resulta más que suficiente para las necesidades del proyecto.

El testing se implementa mediante herramientas especializadas. Jest actúa como framework principal de testing para pruebas unitarias tanto en backend como frontend. Supertest facilita las pruebas de integración de APIs HTTP, permitiendo verificar el comportamiento end-to-end de los endpoints. Playwright opcionalmente puede emplearse para pruebas end-to-end del flujo completo de usuario a través de la interfaz web.

3) *Justificación de la Selección Tecnológica:* La elección del stack tecnológico descrito se fundamenta en criterios rigurosos y bien definidos que garantizan el éxito del proyecto. La madurez y adopción generalizada de todas las tecnologías seleccionadas aseguran que cuentan con documentación extensa y de alta calidad, comunidades activas que proporcionan soporte continuo, y un uso consolidado y probado en entornos de producción reales en la industria. El tipado estático proporcionado por TypeScript en todas las capas del sistema reduce dramáticamente los errores de programación, mejora sustancialmente la experiencia de desarrollo mediante autocompletado y verificación en tiempo real, y facilita enormemente el mantenimiento del código a largo plazo.

La arquitectura modular habilitada por NestJS facilita la implementación de una arquitectura por capas clara, bien organizada y altamente escalable, ideal para demostrar conceptos arquitectónicos fundamentales en un contexto académico. El ecosistema perfectamente integrado entre las herramientas seleccionadas, específicamente la compatibilidad nativa entre Next.js y React, NestJS y TypeScript, y Prisma con PostgreSQL, reduce significativamente la complejidad de integración que típicamente consume tiempo valioso en proyectos multi-tecnología.

El despliegue accesible mediante las plataformas seleccionadas, que ofrecen planes gratuitos con capacidades suficientes para el alcance académico del proyecto, permite demostrar la aplicación completamente funcional en producción sin incurrir en costos de infraestructura, democratizando el acceso a tecnologías de nivel empresarial. Finalmente, la curva de aprendizaje razonable de todas las tecnologías elegidas, respaldada por documentación excepcional y abundantes recursos educativos en forma de tutoriales, videos y cursos, facilita su adopción y comprensión profunda durante todo el ciclo de desarrollo del proyecto.

VII. METODOLOGÍA

El desarrollo del proyecto se llevará a cabo mediante una metodología iterativa e incremental, adaptada a las características de un proyecto académico individual con restricciones de tiempo. Este enfoque permite construir el sistema de forma progresiva, priorizando las funcionalidades esenciales y garantizando que se cumplan los objetivos específicos definidos en el plazo establecido de **12 semanas**.

A. *Enfoque Metodológico*

El proyecto adoptará un modelo de **desarrollo iterativo incremental con enfoque MVP** (Producto Mínimo Viable), que consiste en identificar y desarrollar únicamente las funcionalidades críticas necesarias para demostrar el cumplimiento de los objetivos académicos.

Este enfoque se complementa con **prácticas ágiles adaptadas** para trabajo individual:

- **Sprints de 1 semana:** Ciclos de trabajo cortos con metas concretas y alcanzables
- **Tablero Kanban:** Visualización del flujo de trabajo mediante Trello
- **Priorización estricta:** Enfoque en funcionalidades core, eliminando características opcionales
- **Desarrollo en paralelo:** Trabajo simultáneo en backend y frontend cuando sea posible
- **Documentación continua:** Avance del documento de tesis en paralelo al desarrollo

1) Principios de Desarrollo:

1. **Simplicidad primero:** Implementar la versión más simple que funcione
2. **No sobre-ingenierizar:** Evitar optimizaciones prematuras o arquitecturas complejas innecesarias
3. **Reutilización:** Aprovechar librerías y componentes existentes
4. **Integración temprana:** Probar la comunicación frontend-backend desde la semana 3
5. **Documentación paralela:** Escribir secciones del documento conforme se completan fases

B. Fases del Proyecto

El proyecto se divide en 6 fases principales distribuidas a lo largo de 12 semanas:

1) Fase 1: Análisis, Diseño y Configuración (Semanas 1-2): Actividades:

- Revisión bibliográfica y análisis de antecedentes
- Definición de requisitos funcionales y no funcionales
- Diseño de arquitectura por capas
- Diseño del modelo de base de datos (Diagrama Entidad-Relación)
- Especificación de endpoints de la API REST
- Creación de wireframes y prototipos en Figma
- Setup de proyectos NestJS y Next.js
- Configuración de Docker y bases de datos
- Configuración de repositorio Git

Entregables:

- Documento de requisitos
- Diagrama de arquitectura por capas
- Diagrama ER de la base de datos
- Especificación de API REST
- Wireframes de interfaces principales
- Repositorio configurado

2) Fase 2: Backend Core (Semanas 3-5):Actividades:

- Implementación del módulo de autenticación
- Desarrollo del módulo de inventario completo (productos, categorías, movimientos)
- Implementación del módulo de ventas y clientes
- Desarrollo de endpoints de reportes
- Integración con Prisma ORM
- Documentación automática con Swagger
- Pruebas unitarias de servicios críticos

Entregables:

- API REST funcional
- Documentación Swagger
- Base de datos configurada y migrada
- Pruebas unitarias implementadas

3) Fase 3: Frontend Core (Semanas 6-8):Actividades:

- Implementación del layout y sistema de navegación
- Desarrollo del módulo de autenticación (UI)
- Creación del módulo de inventario (productos, categorías)
- Desarrollo del punto de venta (POS)
- Implementación del módulo de clientes
- Creación del dashboard y reportes
- Desarrollo de componentes reutilizables

- Integración con la API REST

Entregables:

- Interfaz web responsive completa
- Módulos funcionales integrados
- Componentes UI reutilizables
- Sistema de navegación implementado

4) Fase 4: Integración, Pruebas y Refinamiento (Semanas 9-10): Actividades:

- Pruebas de integración de flujos completos
- Pruebas de casos de uso end-to-end
- Corrección de bugs identificados
- Refinamiento de experiencia de usuario
- Implementación de generación de comprobantes
- Optimización de rendimiento
- Ajustes de diseño visual

Entregables:

- Sistema integrado funcionando
- Reporte de pruebas ejecutadas
- Bugs críticos corregidos
- Comprobantes de venta implementados

5) Fase 5: Validación y Despliegue (Semana 11):Actividades:

- Despliegue del frontend en Vercel
- Despliegue del backend en Railway
- Configuración de base de datos en producción
- Validación con usuarios potenciales (2-3 personas)
- Recopilación de feedback
- Implementación de ajustes críticos

Entregables:

- Sistema desplegado y accesible
- Resultados de validación con usuarios
- Reporte de feedback y ajustes realizados

6) Fase 6: Documentación Final (Semana 12):Actividades:

- Redacción del manual de usuario
- Documentación técnica completa
- Finalización del documento de tesis
- Preparación de presentación final
- Capturas de pantalla y diagramas finales
- Revisión y corrección del documento

Entregables:

- Manual de usuario

- Documentación técnica
- Documento de tesis completo
- Presentación final

TABLA I
Cronograma de Ejecución del Proyecto

Fase	Semanas	Horas	Actividades Principales
1. Análisis, Diseño y Configuración	1-2	40h	- Revisión bibliográfica - Definición de requisitos - Diseño de arquitectura y BD - Wireframes en Figma - Setup de proyectos - Configuración Docker
2. Backend Core	3-5	64h	- Módulo de autenticación - Módulo de inventario completo - Módulo de ventas y clientes - Endpoints de reportes - Documentación Swagger - Pruebas unitarias
3. Frontend Core	6-8	64h	- Layout y autenticación - Módulo de inventario (UI) - Punto de venta (POS) - Módulo de clientes - Dashboard y reportes - Componentes reutilizables
4. Integración y Pruebas	9-10	40h	- Pruebas de flujos completos - Pruebas de integración - Corrección de bugs - Refinamiento UX - Generación de comprobantes
5. Validación y Despliegue	11	20h	- Despliegue en producción - Validación con usuarios - Recopilación de feedback - Ajustes críticos
6. Documentación Final	12	20h	- Manual de usuario - Documentación técnica - Documento de tesis completo - Presentación
TOTAL	12 semanas	248h	Sistema completo + Documento

C. Distribución Semanal Recomendada

D. Hitos de Control

- **Semana 2:** Presentación de arquitectura y diseño
- **Semana 5:** Demo de API funcional con Postman
- **Semana 8:** Demo de interfaz completa integrada
- **Semana 11:** Sistema desplegado y funcional
- **Semana 12:** Entrega final del documento

TABLA II
Distribución Semanal de Actividades

Día	Horas	Actividad Sugerida
Lunes	3h	Desarrollo backend/frontend
Martes	3h	Desarrollo backend/frontend
Miércoles	3h	Desarrollo backend/frontend
Jueves	3h	Desarrollo backend/frontend
Viernes	2h	Pruebas + documentación
Sábado	4h	Desarrollo intensivo
Domingo	2h	Revisión semanal + planeación
Total	20h	Semana completa

VIII. PRESUPUESTO

Se ha establecido un presupuesto enfocado en maximizar el valor obtenido de cada inversión, priorizando el uso de herramientas y tecnologías de código abierto que no requieren licencias costosas.

A. *Notas sobre el Presupuesto*

Inversiones (\$50,000):

- Todas las herramientas de desarrollo son gratuitas (código abierto)
- Servicios de despliegue en planes gratuitos suficientes
- Dominio opcional, puede usarse subdominio gratuito

Gastos de Desarrollo (\$270,000):

- Internet necesario para desarrollo y consultas
- Electricidad por uso del equipo de cómputo

Mano de Obra (\$4,920,000):

- Valor estimado del tiempo invertido por el desarrollador
- Honorarios del director de tesis por asesorías

Ahorro respecto a soluciones comerciales:

TABLA III
Presupuesto Estimado del Proyecto

INVERSIONES		
Equipo de cómputo	Laptop/PC para desarrollo (existente)	\$0
Software	Herramientas open-source	\$0
Cloudinary	Plan gratuito (25GB)	\$0
Vercel	Despliegue frontend (gratis)	\$0
Railway	Backend + PostgreSQL (gratis)	\$0
Dominio	Opcional (.com.co)	\$50,000
Subtotal Inversiones		\$50,000

GASTOS DE DESARROLLO		
Internet	3 meses x \$60,000	\$180,000
Electricidad	3 meses x \$30,000	\$90,000
Subtotal Gastos		\$270,000

MANO DE OBRA		
Desarrollador	248 horas x \$15,000/hora	\$3,720,000
Director de tesis	24 horas x \$50,000/hora	\$1,200,000
Subtotal Mano de Obra		\$4,920,000

OTROS GASTOS		
Transporte	Reuniones + validación	\$100,000
Papelería	Impresiones documento	\$30,000
Imprevistos	5 % del total	\$267,500
Subtotal Otros		\$397,500

COSTO TOTAL DEL PROYECTO		\$5,637,500
---------------------------------	--	--------------------

- Licencias de software (IDE, herramientas): ~\$2,000,000
- Hosting/servidores comerciales: ~\$600,000/año
- **Total ahorrado: ~\$2,600,000**

IX. RESULTADOS

Texto de los resultados

A. *Requerimientos del Sistema*

Los requerimientos del sistema son esenciales en cualquier proyecto. En el caso del presente software, es importante contar con un conjunto claro de especificaciones que guíen todo el proceso de desarrollo.

B. *Requerimientos Funcionales*

Los requerimientos funcionales describen las funcionalidades específicas que debe proporcionar el sistema.

C. *Requerimientos No Funcionales*

Los requerimientos no funcionales desempeñan un papel fundamental en el presente proyecto, ya que definen las características y atributos clave que no están directamente relacionados con la funcionalidad del software, pero que son esenciales para su éxito.

D. *Ejemplo de una lista*

- Item 1
- Item 2

E. *Ejemplo de una figura*

Texto que explica la figura



Fig. 1 Logo de la UPB

X. CONCLUSIONES

Texto de las conclusiones del proyecto...

BIBLIOGRAFÍA

- [1] Vercel, Inc., “Next.js Documentation,” 2024. [En línea]. Disponible en: <https://nextjs.org/docs>
- [2] NestJS Contributors, “NestJS Documentation,” 2024. [En línea]. Disponible en: <https://docs.nestjs.com>
- [3] Prisma Data, Inc., “Prisma Documentation,” 2024. [En línea]. Disponible en: <https://www.prisma.io/docs>
- [4] The PostgreSQL Global Development Group, “PostgreSQL Documentation,” 2024. [En línea]. Disponible en: <https://www.postgresql.org/docs>
- [5] Meta Platforms, Inc., “React Documentation,” 2024. [En línea]. Disponible en: <https://react.dev>
- [6] Microsoft Corporation, “TypeScript Documentation,” 2024. [En línea]. Disponible en: <https://www.typescriptlang.org/docs>
- [7] Cloudinary Ltd., “Cloudinary Documentation,” 2024. [En línea]. Disponible en: <https://cloudinary.com/documentation>
- [8] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Doctoral dissertation, University of California, Irvine, 2000.
- [9] I. Sommerville, *Software Engineering*, 10th ed. Boston: Pearson, 2015.